2.5 Arithmetic Operators

[This section corresponds to K&R Sec. 2.5]

The basic operators for performing arithmetic are the same in many computer languages:

```
+       addition
-       subtraction
*       multiplication
/       division
%       modulus (remainder)
```

The `-` operator can be used in two ways: to subtract two numbers (as in `a - b`), or to negate one number (as in `-a + b` or `a + -b`).

When applied to integers, the division operator `/` discards any remainder, so `1 / 2` is 0 and `7 / 4` is 1. But when either operand is a floating-point quantity (type `float` or `double`), the division operator yields a floating-point result, with a potentially nonzero fractional part. So `1 / 2.0` is 0.5, and `7.0 / 4.0` is 1.75.

The *modulus* operator `%` gives you the remainder when two integers are divided: `1 % 2` is 1; `7 % 4` is 3. (The modulus operator can only be applied to integers.)

An additional arithmetic operation you might be wondering about is exponentiation. Some languages have an exponentiation operator (typically `^ or **`), but C doesn't. (To square or cube a number, just multiply it by itself.)

Multiplication, division, and modulus all have higher *precedence* than addition and subtraction. The term ``precedence'' refers to how ``tightly'' operators bind to their operands (that is, to the things they operate on). In mathematics, multiplication has higher precedence than addition, so `1 + 2 * 3` is 7, not 9. In other words, `1 + 2 * 3` is equivalent to `1 + (2 * 3)`. C is the same way.

All of these operators ``group'' from left to right, which means that when two or more of them have the same precedence and participate next to each other in an expression, the evaluation conceptually proceeds from left to right. For example, `1 - 2 - 3` is equivalent to `(1 - 2) - 3` and gives -4, not +2. (``Grouping'' is sometimes called *associativity*, although the term is used somewhat differently in programming than it is in mathematics. Not all C operators group from left to right; a few group from right to left.)

Whenever the default precedence or associativity doesn't give you the grouping you want, you can always use explicit parentheses. For example, if you wanted to add 1 to 2 and then multiply the result by 3, you could write `(1 + 2) * 3`.

By the way, the word ``arithmetic'' as used in the title of this section is an adjective, not a noun, and it's pronounced differently than the noun: the accent is on the third syllable.

Read sequentially: prev next up top